

동적 FAI 측정 기능

개발 계획서 (Dynamic Rule Feature)

목표: P5(점-점 거리/원 지름·중심), P7(선-선 각도) 측정 항목을 소스 수정 없이 INI 파일로 관리

관련 문서

- FAI 모듈 전체 개발자 가이드: [FAI_리팩토링_개발자_가이드.md](#)
- FAI 측정 패턴 정의: [2_0_FAI_측정패턴_정리.md](#)

관련 Git Branch

- *origin/feature/김준호/동적FAI_개발*

목차

#	슬라이드 제목
1	측정 패턴정리
—	개발 일정
2	개발 일정 개요 (3개월)
3	M1 — Model 완성 + Controller 연결
4	M2 — View 기초 + 진입 버튼 연결
5	M3 — View 완성 + 전체 통합 테스트

#	슬라이드 제목
—	TODO 목록
7	TODO 전체 구조 (MVC)
8	TODO — Model
9	TODO — View : 다이얼로그 리소스
10	TODO — View : Dlg.cpp 구현
11	TODO — View : 규칙 추가/삭제 핸들러
12	TODO — Controller : Inspector 연결
13	TODO — Controller : 진입 버튼 + LoadRules 시점
-	최종 개발 Output 및 정리
14	최종 개발 Output
15	정리

측정 패턴정리(1) - 점-점 거리 / 원 지름·중심

FAI 계산 코드에서 반복되는 계산 커널을 P1~P7 으로 정리 했습니다.
(전체 패턴 정의: [2_0_FAI_측정패턴_정리.md](#))

P5 — 점-점 거리 / 원 지름·중심

항목	내용
계산 방식	두 측정점 간 직선 거리, 또는 8포인트 기반 원 지름·중심 산출
핵심 HALCON API	<code>DistancePp(...)</code> / (8포인트 좌표 직접 연산)
대표 FAI 번호	AKC: 146, 147, 148, 149
서브 타입	<code>POINT_DISTANCE</code> (점-점), <code>CIRCLE_8PT</code> (8포인트 원)

예시

측정점 A ————— 측정점 B
DistancePp
→ 두 점 사이 거리

또는

P0	P1	P2	
P7	⊙	P3	← 8개 측정점에서 원 지름/중심 역산
P6	P5	P4	

→ (기존 FAI 코드 루틴 재사용)

측정 패턴정리 (2) - 선-선 각도

P7 — 선-선 각도

항목	내용
계산 방식	두 선분(L1, L2) 사이의 각도를 측정해 degree로 변환
핵심 HALCON API	<code>AngleL1(...)</code> + <code>TupleDeg(...)</code>
대표 FAI 번호	BOI: 10, 38 / ACT: 29
서브 타입	절대각도(<code>AbsAngle=1</code>) / 부호 있는 각도 (<code>AbsAngle=0</code>)

예시

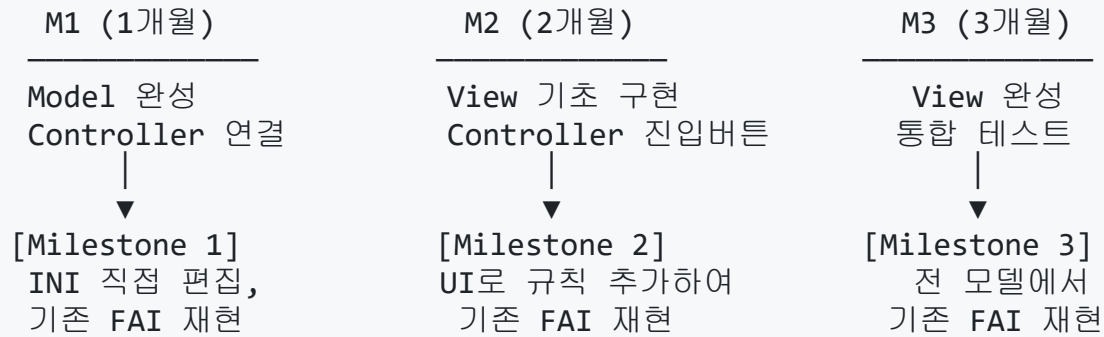
선분 L1: 시작점(Pos0) → 끝점(Pos1)
선분 L2: 시작점(Pos0) → 끝점(Pos1)

$$\begin{array}{c} \text{L1} \\ / \\ / \leftarrow \theta \text{ (AngleL1} \rightarrow \text{TupleDeg)} \\ / \text{----- L2} \end{array}$$

→ θ 값을 FAI 측정값으로 기록

개발 일정

개발 일정 개요 (3개월)



월	주요 작업	Milestone
M1	Model 완성, Controller(Inspector 연결)	AKC 단일 모델 INI 기반 재현
M2	View 기초(리소스·리스트), Controller(진입 버튼)	BOI-AKC UI 기반 재현
M3	View 완성(Add/Del 핸들러), 전체 통합	전 모델(5종) 완전 재현

M1 — Model 완성 + Controller 연결

작업 범위

영역	작업	관련 TODO
Model	ExecuteP5/P7 로직 구현·단위 테스트	TODO — Model
Controller	Inspector 5개에 Execute 호출 삽입	TODO — Controller 1/2
Controller	LoadRules 호출 시점 결정 및 적용	TODO — Controller 2/2

Milestone 1 검증 항목

검증 방법: INI 파일 직접 편집 (UI 없이)

검증 대상: AKC 모델 단일

재현 대상 FAI 항목:

FAI-146: Hole 간 거리 (P5 POINT_DISTANCE)

→ INI에 Rule 정의 후 동적 계산값 == 기존 Inspector 계산값 확인

FAI-147: Hole 원 지름/중심 (P5 CIRCLE_8PT)

→ 8개 측정점 좌표 기반 원 지름 계산 결과 일치 확인

→ 기존 FAI Inspector 루틴과 동일한 알고리즘 재사용

합격 기준: 동적 계산값과 기존 Inspector 값 오차

M2 — View 기초 + 진입 버튼 연결

작업 범위

영역	작업	관련 TODO
View	resource.h ID 상수 + .rc 다이얼로그 레이아웃	TODO — View 다이얼로그 리소스
View	OnInitDialog 컬럼 설정	TODO — View Dlg.cpp
View	RefreshP5List / RefreshP7List 구현	TODO — View Dlg.cpp
Controller	각 TeachFai*Dlg에 진입 버튼 + DoModal 연결	TODO — Controller 2/2

Milestone 2 검증 항목

검증 방법: TeachFaiDlg에서 버튼 클릭 → CTeachFaiDynamicRuleDlg 오픈

검증 대상: BOI, AKC 모델

재현 대상 FAI 항목:

BOI — FAI-38: 기준선-피쳐선 각도 (P7)

→ UI 다이얼로그에서 규칙 확인 가능 (리스트에 표시)

→ INI 저장 후 검사 실행 → 기존 Inspector 값과 일치

AKC — FAI-146, 148: 점-점 거리 (P5)

→ UI에서 규칙 리스트 조회 가능

합격 기준:

- 다이얼로그가 열리고 기존 규칙 리스트를 표시할 것

- INI 저장 → 재검사 결과값 오차

M3 — View 완성 + 전체 통합 테스트

작업 범위

영역	작업	관련 TODO
View	OnBnClickedBtnAddP5/P7 완전 구현	TODO — View 핸들러
View	OnBnClickedBtnDelP5/P7 완전 구현	TODO — View 핸들러
View	PosName() — 포지션 표시명 연결	TODO — ViewDlg.cpp
통합	전 모델(BOI/AKC/ACT/ATW/BOS) 회귀 테스트	—

Milestone 3 검증 항목

검증 방법: UI에서 규칙 추가 → 저장 → 검사 실행 → 기존값 비교

검증 대상: 전 지원 모델 5종 (BOI / AKC / ACT / ATW / BOS)

재현 대상 FAI 항목 (모델별 대표):

BOI — FAI-38 (P7 선-선 각도)

AKC — FAI-146 (P5 점-점 거리), FAI-147 (P5 원 지름·8포인트)

ACT — FAI-29 (P7 선-선 각도)

ATW — P5 / P7 해당 항목 (ATW FAI 정의서 기준)

BOS — P5 / P7 해당 항목 (BOS FAI 정의서 기준)

합격 기준:

- UI로 규칙 추가/삭제/저장이 오류 없이 동작할 것
- 전 모델 동적 계산값 ↔ 기존 Inspector 값 오차
- 앱 재시작 후 INI 로드 → 동일 결과 재현될 것

TODO 목록

TODO — 전체 구조

MVC 구조

[Model]	[View]	[Controller]
FAIDynamicRule.h	TeachFaiDynamic	Inspector
FAIDynamicRuleManager	RuleDlg.cpp	연결부

⚠ 미완성

⚠ 미완성

✗ 미연결

영역	상태	주요 TODO
Model	⚠ 미완성	계산 구현 및 테스트
View(UI)	⚠ 미완성	리스트 표시, 규칙 추가/삭제 UI 구현
Controller	✗ 미연결	Inspector Execute 연결, 진입 버튼 연결, 빌드 등록

TODO — Model, 실제 계산부분

FAIDynamicRule.h / FAIDynamicRuleManager.h/cpp — (초안)

FAIDynamicRule.h

FAIDynamicP5Rule 구조체 (faiId, type, posA/B, idxA/B, multiple, offset)

FAIDynamicP7Rule 구조체 (faiId, posL1[2], posL2[2], idxL1/L2, bAbsAngle, multiple, offset)

FAIDynamicRuleManager.h/cpp

GetInstance() — 싱글턴

LoadRules(modelType) — INI → 메모리

SaveRules(modelType) — 메모리 → INI

AddP5Rule / AddP7Rule

UpdateP5Rule / UpdateP7Rule

RemoveP5Rule / RemoveP7Rule

GetP5Rules / GetP7Rules

ExecuteP5Rules(...) — DistancePp / InnerCircle 계산 + Review XLD 생성

ExecuteP7Rules(...) — AngleL1 / TupleDeg 계산 + Review XLD 생성

TODO — View (1/2) : 다이얼로그 리소스

uScan.rc + resource.h — 리소스 파일에 다이얼로그 정의 추가 필요

```
// resource.h 에 추가
#define IDD_TEACH_FAI_DYNAMIC_RULE_DLG 419
#define IDC_LIST_DYNAMIC_P5_RULES 8628
...
#define IDC_BTN_DYN_CLOSE 8635

// uScan.rc 에 다이얼로그 레이아웃 추가 (대략적인 구조)
IDD_TEACH_FAI_DYNAMIC_RULE_DLG DIALOG ...
    // 상단 영역: P5 규칙 리스트
    GROUPBOX "P5 규칙 (점-점 거리 / 원 지름·8포인트)"
    SysListView32 IDC_LIST_DYNAMIC_P5_RULES
    PUSHBUTTON "추가" IDC_BTN_DYN_ADD_P5
    PUSHBUTTON "삭제" IDC_BTN_DYN_DEL_P5

    // 하단 영역: P7 규칙 리스트
    GROUPBOX "P7 규칙 (선-선 각도)"
    SysListView32 IDC_LIST_DYNAMIC_P7_RULES
    PUSHBUTTON "추가" IDC_BTN_DYN_ADD_P7
    PUSHBUTTON "삭제" IDC_BTN_DYN_DEL_P7

    // 우측: 저장/닫기
    PUSHBUTTON "저장" IDC_BTN_DYN_SAVE
    PUSHBUTTON "닫기" IDC_BTN_DYN_CLOSE
```

TODO — View (2/2) : TeachFaiDynamicRuleDlg.cpp

모두 TeachFaiDynamicRuleDlg.cpp 내부 — FAI 설정 함수들 구현 필요

```
OnInitDialog():
    // P5 리스트 컬럼 설정 (현재 주석처리 상태)
    m_listP5.InsertColumn(0, "FAI#", width=50)
    m_listP5.InsertColumn(1, "Type", width=90) // "POINT_DIST" or "CIRCLE_8PT"
    ...

    // P7 리스트 컬럼 설정 (유사하게)
    m_listP7.InsertColumn(0, "FAI#")
    m_listP7.InsertColumn(1, "L1시작Pos") ... (L1끝, L2시작, L2끝, 절대각, x배율, offset)
    ...

RefreshP5List():
    m_listP5.DeleteAllItems()
    rules = manager.GetP5Rules(m_modelType)
    for i, rule in rules:
        m_listP5.InsertItem(i, FAI 번호 정의)
        m_listP5.SetItemText(i, 1, 인덱스와 계산 방법 정의)
        m_listP5.SetItemText(i, 2, 측정 위치 정의)
    ...

RefreshP7List():
    // P5와 동일한 패턴으로 L1/L2 4개 포인트 표시
```

TODO — View (2/2) : 규칙 추가/삭제 핸들러

```
OnBnClickedBtnAddP5():
// 방법 A (단순): 입력 다이얼로그 띄우기
//   → FAI#, Type(콤보), PosA(콤보), IdxA, PosB(콤보), IdxB, 배열, offset 입력
//   → OK 클릭 시 rule 구성 후 manager.AddP5Rule(modelType, rule)

// 방법 B (UX 좋음): 리스트 끝에 빈 행 추가 후 인라인 편집
//   → 현재 MFC LVS_EDITLABELS 지원 필요
//   → 구현 복잡도 높음 → 우선 방법 A 권장

rule = 사용자 입력값으로 채운 FAIDynamicP5Rule
manager.AddP5Rule(m_modelType, rule)
RefreshP5List()

OnBnClickedBtnDelP5():
sel = m_listP5.GetNextItem(-1, LVNI_SELECTED)
if sel < 0: return // 선택 없음
manager.RemoveP5Rule(m_modelType, sel)
RefreshP5List()

OnBnClickedBtnAddP7():
// P5와 동일한 패턴, 단 L1/L2 각 2점(시작/끝) + AbsAngle 플래그 추가

OnBnClickedBtnDelP7():
sel = m_listP7.GetNextItem(-1, LVNI_SELECTED)
if sel < 0: return
manager.RemoveP7Rule(m_modelType, sel)
RefreshP7List()
```

TODO — Controller (1/2) : Inspector 연결

Inspector 5개 파일에 Execute 호출 삽입

[Inspector 연결] – 5개 파일 동일 패턴

파일: [모델명]FAIInspector.cpp

각 파일 상단 include 추가:

```
#include "../Dynamic/FAIDynamicRuleManager.h"
```

각 Inspect() 함수 끝 (return 직전):

```
auto& dynMgr = FAIDynamicRuleManager::GetInstance()  
dynMgr.ExecuteP5Rules(해당모델_ENUM, pStruct, iBufferIdx, iParamNo, m_pAlgorithm)  
dynMgr.ExecuteP7Rules(해당모델_ENUM, pStruct, iBufferIdx, iParamNo, m_pAlgorithm)
```

TODO — Controller (2/2) : 진입 버튼 + LoadRules 시점

[진입 버튼 연결] - 각 TeachFai*Dlg 에 추가

파일: TeachFaiBOIDlg, TeachFaiAKCDlg, TeachFaiACTDlg, TeachFaiATWDlg, TeachFaiBOSDlg

각 .rc 파일:

```
PUSHBUTTON "동적 규칙 관리", IDC_BTN_OPEN_DYNAMIC_RULE_MGR, x, y, w, h
```

각 .cpp 파일:

```
ON_BN_CLICKED(IDC_BTN_OPEN_DYNAMIC_RULE_MGR, OnBnClickedOpenDynamicRuleMgr)
```

```
OnBnClickedOpenDynamicRuleMgr():
```

```
CTeachFaiDynamicRuleDlg dlg(현재_모델_타입, this)
```

```
dlg.DoModal()
```

[LoadRules 호출 시점 결정]

현재: TeachFaiDynamicRuleDlg::OnInitDialog() 에서만 호출됨

→ Inspect() 실행 전 LoadRules가 호출된 적 없으면 Execute가 빈 규칙 리스트로 동작

권장 추가 위치:

FAIInspectorFactory.cpp의 ReplaceFAIInspector() 내부

→ 모델 전환 시 새 Inspector 생성 직후 LoadRules(newModelType) 호출

또는

각 Inspector 생성자

→ FAIDynamicRuleManager::GetInstance().LoadRules(MODEL_TYPE)

최종 개발 Output 및 정리

최종 개발 Output

모든 FAI 추가/삭제/관리는 GUI(다이얼로그)를 통해 수행

FAI 추가

단계	항목	설명
1	FAI 번호 등록	정수 번호 입력. Sub 넘버링 지원: 예) 147-1 , 147-2
2	검사 방식 등록	4가지 방식 중 선택 (콤보박스) - 우측 표 참조
3	FAI 스펙 등록	이상 판정 기준값(상한/하한) 등록

검사 방식 4종

방식	설명	내부 타입 (예시)
거리	두 측정점 간 직선 거리	P5 , POINT_DISTANCE
각도	두 선분 사이의 각도	P7 , ANGLE
원 반지름/지름	8개 측정점 기반 원 지름·중심 산출	P5 , CIRCLE_8PT
Custom Function	사용자 정의 계산 함수 (확장 예정)	CUSTOM

최종 개발 Output

FAI 삭제

- 리스트에서 항목 선택 → **삭제 버튼** 클릭
- 저장 버튼 클릭 시 INI 파일 반영

리뷰 이미지 및 Log 생성

항목	내용
리뷰 이미지	동적 FAI 측정 결과를 기존 Inspector 결과 이미지와 동일한 경로·포맷으로 저장
Log	기존 FAI Log 형식에 동적 항목 행 추가 (FAI 번호, 측정값, 판정)
Sub 넘버링 표기	Log 및 이미지 파일명에 147-1 , 147-2 형태로 구분

리뷰 이미지 및 Log 생성 흐름 (예시)

[Output 흐름]
GUI에서 규칙 추가/저장
↓
INI 파일 갱신 (자동)
↓
검사 실행 시 ExecuteP5/P7Rules() 호출
↓
측정값 → FAI 결과 버퍼에 기록
↓
리뷰 이미지 생성 (XLD 오버레이)
Log 파일에 해당 FAI 행 추가

정리

개발 계획 요약

1. FAI 규칙은 INI 파일로 관리 — `Data/FaiDynamicRules_{Model}.ini` 에 규칙 정의
2. 각 Inspector 마지막에 Execute 호출 — 소스 수정은 Inspector당 4줄 추가뿐
3. UI는 `CTeachFaiDynamicRuleDlg` — 각 `TeachFai*Dlg`에서 버튼으로 진입
4. P5(점-점/원 지름·8포인트), P7(선-선 각도) 지원 — 타입은 INI의 `Type` 필드로 구분

최종 Output 요약

항목	내용
INI 직접 편집	✗ 불필요 — GUI에서 모든 조작 완결
FAI 추가	GUI에서 번호(Sub 넘버링 포함) · 방식 · 스펙 등록
FAI 삭제	리스트 선택 → 삭제 버튼 → 저장
검사 방식	거리 / 각도 / 원 지름(8포인트) / Custom Function (4종)
리뷰 이미지	기존 Inspector와 동일 경로·포맷으로 자동 저장
Log	기존 FAI Log에 동적 항목 행 추가 (Sub 넘버링 포함)

개정 내역

날짜	작성자	내역	비고
2026/03/16	비전SW 김준호	초안 작성	
2026/03/17	비전SW 김준호	P5 INNER_RADIUS → CIRCLE_8PT(8포인트 기반) 변경 / 최종 개발 Output 섹션 추가 (GUI 기반, Sub 넘버링, 4종 방식, 리뷰이미지/Log)	